

Low-Level Design

Java / Spring Boot Backend

1. Feature modules

```
auth/  
customer/  
application/  
service/  
telemetry/  
incident/  
risk/  
ai/  
audit/  
common/  
config/  
exception/
```

2. Layering

```
Controller → Service → Repository → PostgreSQL  
■  
■→ external client  
■→ Python AI client
```

3. Core classes

Class	Responsibility
Shipment-equivalent: Service	Service master data and SLA configuration
TelemetryService	Validation, persistence and query orchestration
IncidentService	Incident creation/state transitions
RiskAssessmentService	Persist ML risk outputs
AIService	Create AI analysis request and manage result
PythonAIClient	HTTP client to FastAPI
AuditService	Persist auditable events
RateLimitService	Enforce request quotas

4. Transaction boundaries

- Telemetry ingestion transaction validates and persists a valid observation.
- Incident creation and initial status transition are atomic.
- Recommendation approval and audit entry should be atomic where possible.
- AI inference itself should not hold a database transaction open.

5. Error handling

- ValidationException → 400.
- Authentication/authorization → 401/403.
- Resource not found → 404.
- Conflict/duplicate → 409.
- Downstream AI timeout → bounded 5xx/async fallback.
- Unexpected exception → standardized error ID and server log.

6. Coding standards

- DTOs at API boundaries.
- No business logic in controllers.
- Explicit mapper layer.
- Immutable request/response DTOs where practical.
- Configuration via environment/profiles.
- Unit and integration tests per feature.